



INGENIERÍA EN DESARROLLO DE SOFTWARE

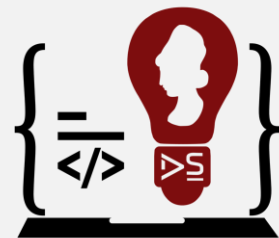
CICLO 1/2025 – ASIGNATURA:

MANEJO DE ESTRUCTURA DE DATOS

UNIVERSIDAD DE EL SALVADOR - FMOcc

Departamento de Ingeniería y Arquitectura

Docente: _____



Ingeniería en Desarrollo
de Software



CASOS PRÁCTICOS DE COMPLEJIDADES

ÓRDENES DE COMPLEJIDAD

Hasta aquí, es posible medir cualquier algoritmo. Pero incluso así es complicado comparar unos algoritmos con otros.

Además, se requiere poder categorizar que tan complejo es el algoritmo de otros. Para esto recurriremos al **orden de complejidad**.



CASOS PRÁCTICOS DE COMPLEJIDADES

ÓRDENES DE COMPLEJIDAD

¡Para medir un algoritmo necesitamos recurrir al escenario de peor caso y con datos demasiado grandes!

Viéndolo de esta manera, las ecuaciones de O Grande podríamos simplificarlas, eliminando algunas características que son irrelevantes.



CASOS PRÁCTICOS DE COMPLEJIDADES

Si comparamos varios algoritmos líneas tales como:

$$O(3n + 1)$$

$$O(20n)$$

$$O(15n + 150)$$

$$O(n)$$

Podemos ver que todo este tipo de ecuaciones podemos agruparlas y referirnos a ellas como Orden Lineal.



CASOS PRÁCTICOS DE COMPLEJIDADES

Constante: $O(1)$

Es la más sencilla y siempre presenta un tiempo de ejecución constante. Ejemplo:

```
function constante( ) {  
    $x =50;  
    $x++;  
    return $x;  
}
```



CASOS PRÁCTICOS DE COMPLEJIDADES

Lineal: $O(x)$

El tiempo crece linealmente mientras crecen los datos. Ejemplo:

```
function lineal ($number) {  
    $result = 0;  
    for($x=0; $x<= $number; $x++) {  
        $result++;  
    }  
    return $result;  
}
```



CASOS PRÁCTICOS DE COMPLEJIDADES

Polinómico: $O(x^c)$, $c > 0$

Son algoritmos más comunes, cuando c es 2 se le llama cuadrático, cuando es 3 se le llama cúbico y en general, polinómico.

Cuando c es muy grande suelen ser muy complicados, estos algoritmos suelen tener bucles anidados.



CASOS PRÁCTICOS DE COMPLEJIDADES

Ejejmplo:

```
function polinomico ($number) {  
    $x = 1;  
    for ($i=0; $i<$number; $i++) {  
        for($j=0; $j< i; $j++) {  
            $x += $j + $i;  
        }  
    }  
    return $x;  
}
```




CASOS PRÁCTICOS DE COMPLEJIDADES

Logarítmico: $O(\log x)$

Estos algoritmos indican que el tiempo es menor que el tamaño de los datos de entrada. No importa indicar la base del logaritmo. No suelen ser muchos.



CASOS PRÁCTICOS DE COMPLEJIDADES

```
Ej. function bin($a,$x,$low,$high){  
  $ans = -1;  
  if( $low==$high) $ans = -1;  
  else{  
    $mid = ( $low + floor( ($high-$low)/2 ) );  
    if ( x < a[mid]){ $ans = bin($a,$x,$low,$mid); }  
    else if( x > a[mid] ){ $ans = bin($a,$x,$mid+1,$high); }  
    else $ans = $mid;  
  }  
  return $ans;  
}
```



CASOS PRÁCTICOS DE COMPLEJIDADES

Exponencial: $O(c^x)$

Es uno de las peores complejidades algorítmicas. Sube demasiado a medida que crece los datos de entrada. Un ejemplo es la solución de Fibonacci. Ejemplo:

```
function exponencial($n) {  
    if (n==1 || n==2) { return 1; }  
    return exponencial($n-1) + exponencial($n-2);  
}
```



EJEMPLOS

Ejemplo: Dos bucles anidados dependientes de n.

```
for(int i = 0; i < n; i++)           O(n)
{
    for(int z = 0; z < n; z++)       O(n)
    {
        if(vector[z] > vector[z + 1]) O(1)
        {
            aux = vector[z];         O(1)
            vector[z] = vector[z + 1]; O(1)
            vector[z + 1] = aux;     O(1)
        }
    }
}
```

Tenemos $O(n) * O(n) * O(1) = O(n^2)$, complejidad cuadrática.



EJEMPLOS

Ejemplo: Bucles donde la evolución de la variable de control es ascendente, no lineal.

```
int c = 1;           O(1)
while(c < n)         O(log n)
{
    if(vector[c] < vector[n])  O(1)
    {
        aux = vector[n];      O(1)
        vector[n] = vector[c]; O(1)
        vector[c] = aux;      O(1)
    }
    c = c*2;           O(1)
}
```

La eficiencia para este caso es: $O(1) * O(\log n) * O(1) = O(\log n)$, complejidad logarítmica.



PROBLEMAS P, NP Y NP-COMPLETO

Se suele decir que el orden de complejidad de un problema es el del mejor algoritmo que se conozca para resolverlo. Estos estudios han llevado a la constatación de que existen problemas muy difíciles, problemas que desafían la utilización de los ordenadores para resolverlos.



PROBLEMAS P, NP Y NP-COMPLETO



Clase P.

Los algoritmos de **complejidad polinómica** se dice que son tratables en el sentido de que suelen ser abordables en la práctica. Estos problemas forman la clase P.

Aquellos problemas de complejidad superior a la polinómica, se dice que son **problemas intratables**.



PROBLEMAS P, NP Y NP-COMPLETO

Clase NP.

A estos problemas intratables pueden aplicarse un algoritmo polinómico para comprobar si una posible solución es válida o no.

Los problemas de esta clase se denominan **NP** (la **N** de **no deterministas** y la **P** de **polinómicos**).



PROBLEMAS P, NP Y NP-COMPLETO



Clase NP Completo

Son los peores problemas posibles de clase NP. Estos problemas son todos "iguales" en el sentido de que si se descubre una solución P para alguno de ellos, esta solución sería fácilmente aplicable a todos ellos.

Hay un premio de prestigio equivalente al Nobel reservado para el que descubra la solución.



¿Alguna pregunta?

¡GRACIAS!

